

Deep Research Swarm — session report

Generated: 2026-05-11T06:14:18Z

Session

Session ID: 8b800d4b-a254-4da5-a356-c880dcb36d74

Status: completed

Created: 2026-05-11T05:51:33.247851Z

Wall-clock duration: 19:29

Total cost (USD): 0.000000

Agent invocations (counter): 26

Research question

What are the main trade-offs between synchronized blocks vs explicit locks in java?

Research answer & report

Based on the provided text, here are some key takeaways and insights:

Synchronization vs Locks

Synchronized blocks and explicit locks (like ReentrantLock) are used for synchronization in Java. Both methods can be used to achieve thread safety, but they have different trade-offs.

Trade-Offs

Synchronized Blocks:

Pros: Easy to use, no need to manually manage locks.

Cons: Can lead to deadlocks if not used carefully, and other threads can mutate the object while in the synchronized block.

Explicit Locks (ReentrantLock):

Pros: More flexible, allows for more fine-grained control over locking and unlocking.

Cons: Requires manual management of locks, which can be error-prone.

Best Practices

Use synchronized blocks when you need to protect a small scope of code that doesn't require fine-grained control over locking.

Use explicit locks (ReentrantLock) when you need more control over locking and unlocking, or when you're working with complex synchronization scenarios.

Deadlocks

Deadlocks can occur when two or more threads are waiting for each other to release a resource.

Synchronized blocks can lead to deadlocks if not used carefully.

Atomic Variables

Atomic variables (like AtomicInteger) provide thread-safe updates without the need for explicit locking.

They are useful when you need to update shared data in a thread-safe manner, but don't require fine-grained control over locking.

Conclusion

In summary, both synchronized blocks and explicit locks can be used for synchronization in Java. Synchronized blocks are easy to use but may lead to deadlocks if not used carefully. Explicit locks provide more flexibility but require manual management of locks. Atomic variables offer a thread-safe way to update shared data without the need for explicit locking.

Trust Level

The provided text has a moderate trust level (62/100) due to its reliance on research notes and academic sources, which may have varying levels of credibility. However, the insights and takeaways can still be useful as factual material, without quoting or labeling the specific source blocks.

> Citation review for claim "Research note tied to retrieved source Reading 22: Locks and Synchronization - MIT (<https://web.mit.edu/6.031/www/sp20/classes/22-locks/>) for «What are the main trade-offs between synchronized blocks vs explicit locks in java?»: Even wh..." — similarity min=0.65 (threshold 0.7). Flags: LOW_CITATION_ALIGNMENT

Search wave questions

Same wording as expanding "Search wave" on the graph (sub-questions from SSE tool_call events; export loads full history from the API).

Question	Search rails / hits
synchronized block disadvantages	search_academic · 5 hits
thread safety vs performance	search_academic · 5 hits
performance comparison	search_web · 10 hits
synchronized block vs explicit lock in multithreaded environment	search_code · 8 hits
explicit lock advantages	search_code · 8 hits
explicit lock disadvantages	search_code · 6 hits
synchronized block advantages	search_web · 9 hits
What are some common pitfalls or edge cases when using synchronized blocks versus explicit locks?	search_web · 9 hits
Are there any alternative synchronization mechanisms that can be used in place of synchronized blocks or explicit locks?	search_web · 12 hits
Please provide more information on the advantages and disadvantages of synchronized blocks vs explicit locks in Java.	search_web · 12 hits
How do the performance implications of using synchronized blocks versus explicit locks impact multithreaded applications?	search_web · 12 hits
Can you clarify how the trade-offs between synchronized blocks and explicit locks apply to specific use cases?	search_web · 12 hits
Be aware of mutual exclusion issues when using synchronized blocks.	search_web · 10 hits
Use AtomicReference or AtomicLong for thread-safe updates and comparisons.	search_web · 12 hits
Consider alternative synchronization mechanisms, such as ReentrantLock or java.util.concurrent.locks.	search_web · 12 hits
Consider using ReentrantLock for more fine-grained control over locking and unlocking.	search_web · 11 hits
Review the trade-offs between performance and complexity when choosing between synchronized blocks and explicit locks.	search_web · 12 hits

Intermediate statement breakdowns

Planner sub-questions matched to search-wave summaries when available; numbered rows are critic follow-ups.

Question	Summarized response
synchronized block advantages	Here is a 200-500 token summary answering the sub-question "synchronized block advantages": Synchronized blocks offer several advantages over synchronized methods, including: They do not require explicit synchronization of entire code blocks, making them more efficient. They provide mutual exclusion for shared resources in multithreaded programs. They can be used to ensure thread safety by locking specific objects. In contrast, synchronized methods are often used as a last resort or when direct synchronization is necessary. However, they may not always be the most efficient solution due to their overhead and potential performance issues. A Java Synchronized Block is typically used instead of a synchronized method when: Direct synchronization is required. The code block needs to be executed in a specific order....
synchronized block disadvantages	Here is a 200-500 token summary answering only the sub-question "synchronized block disadvantages": The synchronized block in particle physics has several disadvantages. Firstly, it relies on analyticity constraints used in Landshoff-Donnachie fits, which may not be sufficient to accurately describe the energy behavior of hadron-hadron cross sections. The implications from these constraints are discussed in detail. Secondly, the gravitational torque of bars in galaxies is determined by transforming light distributions into potentials and deriving maximum ratios of tangential forces relative to radial forces. However, this method has limitations, as it assumes a specific form for the potential and may not accurately capture the complex dynamics of galaxy evolution. Thirdly, new analyticity constraints on high-energy hadron-hadron cross sections have been proposed,...
explicit lock advantages	Here is a 200-500 token summary answering the sub-question "Explicit lock advantages" with conflicts noted: Web Locks API suggests using explicit locking for untrusted web content to prevent timing attacks. FilipSnastins' concurrency-control-with-dynamodb example shows that explicit locking can be used in PostgreSQL, but requires careful consideration of database features. Hibernate-ORM recommends pessimistic locking as a default approach, which may not be suitable for all use cases. Spirit's migrate.md document notes that explicit locks can cause excessive locking and should be avoided unless necessary. The possibly excessive locking issue

Question	Summarized response
	suggests that explicit locks can lead to high performance but may also introduce lock contention. RouteInfoManager's feature suggests using segmented locking for improved concurrency and consistency, with a 93.75% reduction in lock...
explicit lock disadvantages	Here is a 200-500 token summary answering the sub-question "Explicit Lock Disadvantages": Explicit locks are typically required for systems that require isolation (as in ACID) and concurrency control, such as databases. However, they can have several disadvantages. They often require explicit locking of objects, which can be inconvenient for applications that need to perform multiple operations on the same data simultaneously. Explicit locks can lead to slower reads and compaction processes in certain data structures like LSM-Trees and B-Trees. * Optimistic locking is a technique used by some systems (like PostgreSQL) that allows transactions to proceed even if some data has been modified since they started. However, explicit locking can be more efficient than optimistic locking for certain use cases....
performance comparison	Performance Comparison The provided excerpts compare various CPUs from different manufacturers across different benchmarks. Here's a summary of the key findings: The PassMark CPU Benchmark (https://www.cpubenchmark.net/singleCompare.php) compares up to 5 different CPUs, including Intel Core Ultra 7 270K Plus, Intel Core Ultra 9 285K, Apple M2 8 Core 3500 MHz, and AMD Ryzen AI 9 HX 370. The CPU UserBenchmarks - 1424 Processors Compared (https://cpu.userbenchmark.com/) provides a comprehensive comparison of 1424 processors from various manufacturers, including Intel Core i7-11700K, AMD Ryzen 9 5900X, and Apple M1 Pro. The CPU Comparison: Performance Tests – NanoReview (https://nanoreview.net/en/cpu-compare) compares desktop processors from different manufacturers, including Apple M5 Pro, Apple M5 Max, and Intel Core Ultra X9 388H. Puget Bench Compare Results (https://www.pugetsystems.com/pugetbench/results/compare/) provides...
thread safety vs performance	Here is a 200-500 token summary answering the sub-question "thread safety vs performance" based on the provided sources: Thread safety and performance are two competing concerns in software systems that continuously evolve due to new functionalities or maintenance activities. On one hand, multi-objective algorithms have shown the ability to discover alternatives by pursuing different objectives simultaneously. However, their high computational complexity may lead to poor performance if not carefully designed (Source: Performance of Genetic Algorithms in the Context of Software Model Refactoring). On the other hand, a modern thread pool implementation can achieve high-performance scientific computing while providing precise low-level control over thread details. This approach is particularly useful for tasks that require fine-grained parallelism and minimal overhead (Source: A...
synchronized block vs explicit lock in multithreaded environment	Here is a 200–500 token summary answering the sub-question "synchronized block vs explicit lock in multithreaded environment" based on the provided sources: The main difference between synchronized blocks and explicit locks lies in their purpose and usage. Synchronized blocks are used to synchronize access to shared resources, ensuring that only one thread can execute a particular section of code at a time. They hold a lock, which allows for exclusive access. In contrast, explicit locks, such as lock or synchronized, require manual code to acquire and release the lock. This approach is more verbose and error-prone compared to synchronized blocks. The GitHub repository "Multi-threading-in-Java" by ZhianMai provides an example of using synchronized blocks in a multithreaded environment, where all locking...
Follow-up sub-question #1	Based on the provided sources, here is a 200-500 token summary answering the sub-question: Synchronized blocks are used to prevent multiple threads from executing a portion of code in a method at the same point in time. They provide better performance compared to explicit locks, especially when the entire method needs to be synchronized. On the other hand, explicit locks (e.g., lock(), unlock()) offer more flexibility and advanced features, such as fairness and tryLock methods, which can improve thread safety. A Java Synchronized Block is a block of code that prevents multiple threads from executing it simultaneously. It provides better performance compared to explicit locks but has limitations in terms of control over lock management. The main advantages of using...
Follow-up sub-question #2	Based on the provided sources, here is a 200-500 token summary answering the sub-question: The trade-offs between synchronized blocks and explicit locks in Java depend on the specific use case. Synchronized blocks are simpler to use for basic locking needs but may not provide sufficient control over concurrency. Explicit locks, introduced in Java 1.5, offer greater flexibility and advanced features, such as support for read-write locks. In general, if you're using java.util.concurrent.locks, it's recommended to avoid using synchronized blocks due to their limited functionality. Instead, use explicit locking mechanisms like lock.lock() and lock.unlock(). This approach provides better control over concurrency and can improve performance in certain scenarios. The key differences between synchronized blocks and explicit locks lie in their capabilities...
Follow-up sub-question #3	Here is a 200-500 token summary answering the sub-question: The performance implications of using synchronized blocks versus explicit locks in multithreaded applications can vary depending on the specific use case and implementation. Synchronized blocks, which acquire a lock before accessing shared resources, are generally faster than explicit locks because they bypass the JVM's built-in support for acquiring an object's lock and exception handling. However, there is evidence to suggest that synchronized blocks may not always be the best choice due to contention issues when multiple threads try to acquire the same lock simultaneously. This can lead to performance degradation and even kill parallelism. Explicit locks, on the other hand, are more performant because they manage the locking status in a...

Question	Summarized response
Follow-up sub-question #4	Based on the provided sources, here are some common pitfalls or edge cases when using synchronized blocks versus explicit locks: 1. Mutual Exclusion: Using synchronized blocks can lead to mutual exclusion issues if not used carefully, as only one thread can execute a block guarded by the same monitor object at a time. 2. Visibility: The visibility of a lock can be affected by how it is used in conjunction with other synchronization mechanisms, such as ReentrantLock. 3. Hard to Maintain: Using string literals as locks makes it difficult to track where and why certain locks are being acquired, leading to maintenance challenges. 4. Less Concurrency Due to Locking the Entire Method or Class: Using a lock can reduce concurrency...
Follow-up sub-question #5	Based on the provided sources, alternative synchronization mechanisms that can be used in place of synchronized blocks or explicit locks include: 1. AtomicReference: This class provides thread-safe methods for updating and comparing references. 2. AtomicLong: Similar to AtomicReference, this class offers thread-safe operations for updating and comparing long values. 3. ReentrantLock: As mentioned in the YouTube video, ReentrantLock is an advanced alternative to synchronized blocks that allows for more fine-grained control over locking and unlocking. 4. java.util.concurrent.locks.ReentrantLock: This is a built-in class provided by Java's concurrency package that offers similar functionality to ReentrantLock. These alternatives offer improved performance, flexibility, and thread safety compared to traditional synchronized blocks or explicit locks.
Follow-up sub-question #6	Here is a 200–500 token summary answering the sub-question about using ReentrantLock: ReentrantLock can be used to achieve more fine-grained control over lock behavior compared to synchronized constructs, which require explicit block structure and may not provide interruptible locking. However, its flexibility comes at the cost of requiring explicit lock/unlock calls, such as try-finally blocks. In contrast, ReentrantLock provides a more structured approach to concurrency, allowing for better synchronization in multithreading environments. Its ability to hold locks across method calls is particularly useful when non-blocking behavior is required. However, ReentrantLock's flexibility also means that it can be less efficient than synchronized constructs in certain scenarios, such as when the same lock is held multiple times within a single thread. Additionally,...
Follow-up sub-question #7	Here's a 200–500 token summary answering the sub-question: Mutual exclusion issues when using synchronized blocks are crucial to understand, particularly in scenarios where multiple threads need to access shared resources simultaneously. The prevailing method of mutual exclusion is to use a mutex (mutual exclusion) where each thread must take its turn accessing the shared resource. This ensures that no two threads can access the resource at the same time, preventing race conditions and ensuring safety. However, using synchronized blocks without wait() or compareAndSwap can lead to busy waiting, which may not be efficient in all cases. Synchronizing threads effectively requires a deep understanding of synchronization primitives like mutexes, semaphores, test-and-set, spinlocks, and blocking. These mechanisms can help ensure mutual exclusion...
Follow-up sub-question #8	Here is a 200–500 token summary answering the sub-question "Use AtomicReference or AtomicLong for thread-safe updates and comparisons" with conflicts noted where sources disagree: To ensure thread safety when updating shared values in Java, consider using either AtomicReference or AtomicLong. Both classes provide atomic operations that can resolve race conditions. However, there is a conflict between the two approaches. The article by OpenLiberty.io suggests using AtomicReference alone for thread-safe updates and comparisons (https://openliberty.io/blog/2021/09/21/building-on-atomic-reference.html). This approach can be used for simple cases where multiple threads need to update a shared value without the need for explicit locks. On the other hand, the article by Stack Overflow suggests using AtomicLong instead of AtomicReference when you need to compare and set values (https://stackoverflow.com/questions/37716360/why-do-we-need-to-compare-and-set-when-set-is-already-atomic-in-java)....
Follow-up sub-question #9	Based on the provided sources, here is a 200-500 token summary answering the sub-question: Synchronized blocks and explicit locks offer different trade-offs between performance and complexity. Synchronized blocks are generally faster due to their low overhead, but may lead to deadlocks if not used carefully. Explicit locks, such as ReentrantLock, provide mutual exclusion but require more code and can be less efficient in certain scenarios. In terms of ease of use, synchronized blocks are often preferred for simple use cases, while explicit locks are better suited for complex scenarios where thread safety is critical. However, the choice between the two ultimately depends on the specific requirements of the application. The sources also highlight that Java Performance: synchronized() vs Lock ...
Follow-up sub-question #10	Here are 200-500 token summaries answering the sub-question "Consider alternative synchronization mechanisms, such as ReentrantLock or java.util.concurrent.locks" with conflicts noted: 1. ReentrantLock vs Synchronized: Both provide mutual exclusion but ReentrantLock offers more control and features like timed locking and non-blocking attempts. However, synchronized is a JVM-managed object that can perform fancy things like lock escalation and de-escalation. 2. Comparison of Synchronized and ReentrantLock performance: A study found that ReentrantLock outperforms synchronized in certain scenarios due to its ability to handle high-frequency read operations compared to write operations. 3. ReentrantLock as a complete replacement for synchronization: A question on Stack Overflow suggests that ReentrantLock is not a complete replacement for synchronization, but rather an alternative tool with similar functionality. 4. Understanding...

Parameter	Value
model_fast	ollama/llama3.2:1b
model_strong	ollama/llama3.2:3b
embedding_model	ollama/nomic-embed-text
similarity_mode	tfidf
session_cost_limit_usd	0
max_parallel_agent_calls	10

Agent steps (wall time per node)

Agent id	Label	Duration (s)	Outcome
planner	Planner	24.42	ok
parallel_search	Parallel Search	238.98	ok
critic	Critic	72.94	ok
critic_route_prepare	Critic router	0	ok
parallel_search	Parallel Search	161.7	ok
critic	Critic	56.59	ok
critic_route_prepare	Critic router	0	ok
parallel_search	Parallel Search	156.8	ok
critic	Critic	61.56	ok
extract_claims	Extract Claims	182.97	ok
fact_check	Fact Check	42.18	ok
synthesize	Synthesize	171.3	ok
citation_format	Citation formatter	0.02	ok

Verified claims & trust

Trust (band / score)	Claim (excerpt)	Cited / verified sources
MODERATE (62)	Research note tied to retrieved source [src_b9e218c7f2b8eca5] for «What are the main trade-offs between synchronized blocks vs explicit locks in java?»: Even while t is in its synchronized block, another thread can dangerously mutate the object, simply by neglecting to use synchronized itself. In order to use an	Reading 22: Locks and Synchronization - MIT — https://web.mit.edu/6.031/www/sp20/classes/22-locks/

Source catalog (URLs from graph state)

Source id	Title	URL
src_981ae27089715cf5	Advantages of [Synchronized Block] over [Synchronized Method]	https://medium.com/@codingsprint/advantages-of-synchronized-block-over-synchronized-method-c2f04449db3a
src_0a965e41b5c73198	java - Is there an advantage to use a Synchronized Method instead ...	https://stackoverflow.com/questions/574240/is-there-an-advantage-to-use-a-synchronized-method-instead-of-a-synchronized-blo
src_3fd564e8753378ce	Concurrency in Java: Mastering Synchronized Blocks for Thread ...	https://www.linkedin.com/pulse/concurrency-java-mastering-synchronized-blocks-thread-elham-moharrami-61bsf
src_e9891395f44ad2ec	What is a Java Synchronized Block? Examples & Advantages	https://blog.fastthread.io/java-synchronized-block/
src_8f249f753603727f	Why Most Java synchronized Blocks Are a Performance Trap (And ...	https://medium.com/@reallyouttaworld/why-most-java-synchronized-blocks-are-a-performance-trap-and-what-to-use-instead-12a67f6a29be
src_893ed7869a27f186	Java Method and Block Synchronization - GeeksforGeeks	https://www.geeksforgeeks.org/java/method-block-synchronization-java/
src_f5c25705932c137a	In Java, what happens under the hood when we use synchronized ...	https://www.quora.com/In-Java-what-happens-under-the-hood-when-we-use-synchronized-block-method-How-does-synchronized-keyword-work-internally
src_ec8af8d55b09c26c	synchronized method vs synchronized block having same number of ...	https://www.reddit.com/r/learnjava/comments/nnkxrb/synchronized_method_vs_synchronized_block_having/

Source id	Title	URL
src_3d7fa051b4beecf2	Synchronized vs ReentrantLock Concurrency Interview - AlgoMaster	https://algomaster.io/learn/concurrency-interview/java-synchronized-vs-reentrantlock
src_85954d986f3fdd9b	Implications from analyticity constraints used in a Landshoff-Donnachie fit	https://arxiv.org/abs/hep-ph/0605216v1
src_3cf3b8cfc0a98dd	The gravitational torque of bars in optically unbarred and barred galaxies	https://arxiv.org/abs/astro-ph/0106019v1
src_f59dd889df4f5bf9	New analyticity constraints on the high energy behavior of hadron-hadron cross sections	https://arxiv.org/abs/hep-ph/0601210v3
src_5d553da2bc729a51	Fock spaces and refined Severi degrees	https://arxiv.org/abs/1409.4868v1
src_c3f8be1a5c400381	A new numerical method for obtaining gluon distribution functions $G(x, Q^2) = xg(x, Q^2)$, from the proton structure functi	https://arxiv.org/abs/0907.4790v1
src_04a1ba968065bb6b	Web Locks API #217 - w3ctag/design-reviews - GitHub	https://github.com/w3ctag/design-reviews/issues/217
src_1074adbb70265c7d	filipsnastins/concurrency-control-with-dynamodb: Examples ... - GitHub	https://github.com/filipsnastins/concurrency-control-with-dynamodb
src_77f7d8e2b083ec37	Locking - hibernate-orm - GitHub	https://github.com/hibernate/hibernate-orm/blob/master/documentation/src/main/asciidoc/userguide/chapters/locking/Locking.adoc
src_bc24cdee2bae33ee	spirit/docs/migrate.md at main · block/spirit · GitHub	https://github.com/block/spirit/blob/main/docs/migrate.md
src_879ab7c934531c9b	Possibly excessive locking in the upsert function · Issue #296 - GitHub	https://github.com/JasperFx/marten/issues/296
src_2b7f173599139236	[Feature] Implement segmented locking in RouteInfoManager for ...	https://github.com/mxsm/rocketmq-rust/issues/4279
src_21a38a10eb4f50b4	decoupling stable CI and incompatibilities discovery (let's use lock ...	https://github.com/astropy/astropy/issues/17596
src_face2fde3b194fdb	Multiprocess compatible FileSink using file locks #1257 - GitHub	https://github.com/Delgan/loguru/issues/1257
src_e761b29ab5c3b20b	The Systems Engineer Interview - GitHub Gist	https://gist.github.com/seanpianka/55269b93114ebbc03270e62431c40697
src_6217a2ecd6148f1c	ahmedhammad97/Designing-Data-Intensive-Applications -Notes	https://github.com/ahmedhammad97/Designing-Data-Intensive-Applications -Notes
src_a53fcd48e8c6aa0b	gino/docs/explanation/why.rst at master · GitHub	https://github.com/python-gino/gino/blob/master/docs/explanation/why.rst
src_64430f13b90f5af5	Why NH encourage user to using transaction #2999 - GitHub	https://github.com/nhibernate/nhibernate-core/discussions/2999
src_efe9a0e8e0e18988	PassMark - CPU Comparison	https://www.cpubenchmark.net/singleCompare.php
src_7db1fef75b63df9f	CPU UserBenchmarks - 1424 Processors Compared	https://cpu.userbenchmark.com/
src_20c323bee84b5353	CPU Comparison: Performance Tests – NanoReview	https://nanoreview.net/en/cpu-compare
src_34e26514ac20a299	Puget Bench Compare Results	https://www.pugetsystems.com/pugetbench/results/compare/
src_c01488e65e8b58a2	Geekbench 6 - Cross-Platform Benchmark	https://www.geekbench.com/
src_6abcbf072440f020	Unbiased hardware comparisons	https://technical.city/en
src_59f7904544d70e64	Trusted source to compare CPU's? : r/buildapc - Reddit	https://www.reddit.com/r/buildapc/comments/1i4gww3/trusted_source_to_compare_cpus/
src_99c5003caa026f7f	PassMark PerformanceTest - PC benchmark software	https://www.passmark.com/products/performancetest/?srsltid=AfmBOorqBM4JU1sm3B73hIJA1i488x-mLhALyl_Je2Ea31vbNslD5pwn
src_e346b879d4fb3392	PassMark - Videocard Comparison - Video Card Benchmarks	https://www.videocardbenchmark.net/singleCompare.php
src_97174bfe43b83b6a	PassMark PerformanceTest - PC benchmark software	https://www.passmark.com/products/performancetest/?srsltid=AfmBOoq6zXwQblc3SNFRTYfJ-Q4VOOjTmEYjHdmaJ1GNXhdPiHpzNGOa
src_6334afe96b201099	Performance of Genetic Algorithms in the Context of Software Model Refactoring	https://arxiv.org/abs/2308.13875v1
src_36f76b28b3f	Laser pointer prohibition: improving safety or driving	https://arxiv.org/abs/1406.4924v1

Source id	Title	URL
a5352	misclassification	
src_d64293c4d4479ad8	Relating System Safety and Machine Learnt Model Performance	https://arxiv.org/abs/2507.20135v1
src_faf137945edf754	A C++17 Thread Pool for High-Performance Scientific Computing	https://arxiv.org/abs/2105.00613v5
src_9edaf659ddca30c8	Global Thread-Level Inference for Comment Classification in Community Question Answering	https://arxiv.org/abs/1911.08755v1
src_16e9222f420dcaae	ZhianMai/Multi-threading-in-Java - GitHub	https://github.com/ZhianMai/Multi-threading-in-Java
src_37d203f93ef074ec	Multi-threaded server example for synchronization #1381 - GitHub	https://github.com/boostorg/beast/issues/1381
src_dc976e516aed4c11	Java-Concurrency-in-Practice/Chap 3. Sharing Objects/3.1 Visibility ...	https://github.com/XQQ8765/Java-Concurrency-in-Practice/blob/master/Chapter%203.%20Sharing%20Objects/3.1%20Visibility.txt
src_5ef6a3e30895e98d	GitHub - mratsim/weave: A state-of-the-art multithreading runtime	https://github.com/mratsim/weave
src_70b935b5ccf64c94	Java concurrency patterns for educational purposes - GitHub	https://github.com/alxkm/java-concurrency-patterns
src_411e5f843ec34084	Parallel Foreach Thread locking - Issue #13174 - GitHub	https://github.com/PowerShell/PowerShell/issues/13174
src_8e2baa48e41a6f7e	Shared Memory Multithreading for Dart - GitHub	https://github.com/dart-lang/language/blob/main/working/333%20-%20shared%20memory%20multithreading/proposal.md
src_47d5786a2701ff33	Make synchronized block around a cached query optional ... - GitHub	https://github.com/spring-projects/spring-data-jpa/issues/1199
src_8c55929fde19787d	Comparison of Synchronized and ReentrantLock performance in Java	https://www.reddit.com/r/java/comments/1gmeeny/comparison_of_synchronized_and_reentrantlock/
src_e4f3cff5d29b61e0	java - Locking with synchronized block vs Explicit Locking	https://stackoverflow.com/questions/76724780/locking-with-synchronized-block-vs-explicit-locking
src_672472727f6fe4b1	Java Lock Framework vs Thread Synchronization - GeeksforGeeks	https://www.geeksforgeeks.org/java/lock-framework-vs-thread-synchronization-in-java/
src_63d03e72be51857	Difference between synchronised block and lock object in JAVA	https://medium.com/@gaganjain9319/difference-between-synchronised-block-and-lock-object-in-java-262219646b5f
src_7ffa10eb7bf12a3f	What is Lock in Java Lock vs synchronized Avoid Deadlocks	https://www.youtube.com/watch?v=Tsvnsr-G6mU
src_47d8f5ece9a0612a	Pros and Cons of Lock (java.util.concurrent.locks) over synchronized ...	https://www.devinline.com/2015/10/Lock-Vs-synchronized-in-java.html
src_a7e28bfb6a2cfc40	Synchronized vs Lock Which One Should You Use? - YouTube	https://www.youtube.com/shorts/mBd0rhcFUeE
src_f80924380dc5a15	In Java, when using locks and synchronize mixed to lock an object ...	https://www.zhihu.com/en/answer/186507889
src_29a168591ba340ad	When should one use concurrent Java data structures, versus ...	https://www.quora.com/When-should-one-use-concurrent-Java-data-structures-versus-explicit-synchronization
src_f696fd9d066a4cd4	Synchronization vs Lock Implementations - YouTube	https://www.youtube.com/watch?v=i7cnZRVbxkE
src_d00fa6029295824d	java - Synchronization vs Lock - Stack Overflow	https://stackoverflow.com/questions/4201713/synchronization-vs-lock
src_18955c5c70acdb4b	What are the key differences between synchronized and Lock?	https://medium.com/@a-marian/what-are-the-key-differences-between-synchronized-and-lock-6038dccb1802
src_7f6c40f069012f5b	Ø>Yà What's the Difference Between and Synchronized Block in Java?	https://medium.com/@idiotN/whats-the-difference-between-synchronized-method-and-synchronized-block-in-java-ab8dfb8c3bd9
src_5e3d3ed49edafe32	Synchronized Blocks and Synchronized Methods in Java - Medium	https://medium.com/@vino7tech/synchronized-blocks-and-synchronized-methods-in-java-2488cb302185
src_b9e218c7f2b8eca5	Reading 22: Locks and Synchronization - MIT	https://web.mit.edu/6.031/www/sp20/classes/22-locks/
src_edfb5805367dae46	Synchronized vs Lock performance	http://blog.vanillajava.blog/2011/07/synchronized-vs-lock-performance.html
src_b21cf009c474818b	Java Performance: synchronized() vs Lock by LiveRamp - Medium	https://medium.com/liveramp-engineering/java-performance-synchronized-vs-lock-301130e62f47
src_c640df0890f3e018	Performance comparison between synchronized and ReentrantLock in Java	https://medium.com/@umeshcapg/performance-comparison-between-synchronized-and-reentrantlock-in-java-2d1148d5f825

Source id	Title	URL
src_c9ee66819143d80c	ReentrantLock vs. synchronized in Java - Naveen Metta	https://naveen-metta.medium.com/reentrantlock-vs-synchronized-in-java-a-comprehensive-comparison-26cf05819cf2
src_552bc2e876a90a44	How the Java "synchronized" Keyword Can Slow Down Your ...	https://medium.com/@niteshthakur498/how-the-java-synchronized-keyword-can-slow-down-your-application-if-not-used-carefully-dc586f86cb30
src_2f1cb91e40959739	Synchronized method + Synchronized block VS Performance	https://coderanch.com/t/521747/java/Synchronized-method-Synchronized-b-lock-Performance
src_7d38b00dc0426c9a	Synchronized But Still Broken: Concurrency Pitfalls Every Java ...	https://codefarm0.medium.com/synchronized-but-still-broken-concurrency-pitfalls-every-java-engineer-should-know-cb17fe586fa0
src_b1dea95291f864f7	Reading 23: Locks and Synchronization - MIT	https://web.mit.edu/6.005/www/fa15/classes/23-locks/
src_c25693e71dc3d3f0	Is there any alternative for synchronization? - CodeRanch	https://coderanch.com/t/449746/java/alternative-synchronization
src_97b916ccb531415	Master Java's Lock Interface: Alternative to synchronized blocks	https://medium.com/@ujjawalr/java-lock-interface-b2f0a4027dba
src_3de4bf9e72e459d5	Alternative for synchronized block in java - Stack Overflow	https://stackoverflow.com/questions/11649473/alternative-for-synchronized-block-in-java
src_124033a922fe0712	ReentrantLock vs Synchronized Which is BEST in Java - YouTube	https://www.youtube.com/watch?v=wTMHRf-qdPM
src_a1ebe02206544047	Let's Replace the synchronized Keyword - Foojay.io	https://foojay.io/today/lets-replace-the-synchronized-keyword/
src_132924c3564f1e69	Java Synchronization Mechanisms — A Practical and Conceptual ...	https://medium.com/@tejaswi555ponnamanda/java-synchronization-mechanisms-a-practical-and-conceptual-guide-0749868b3a47
src_dbd243608e413e9b	Difference Between synchronized and ReentrantLock in Java. Java ...	https://medium.com/@alxkm/difference-between-synchronized-and-reentrantlock-in-java-09402b06c47f
src_7e6a4ae65c1c5475	Why use a ReentrantLock if one can use synchronized(this)?	https://codemia.io/knowledge-hub/path/why_use_a_reentrantlock_if_one_can_use_synchronizedthis
src_74c122176f037986	Unlocking the Mysteries of Java Concurrent Programming PixelsTech	https://www.pixelstech.net/article/1684758352-deep-understanding-of-reentrantlock%3A-unlocking-the-mysteries-of-java-concurrent-programming
src_31dcac1f219518c2	java - Why use a ReentrantLock if one can use synchronized(this)?	https://stackoverflow.com/questions/11821801/why-use-a-reentrantlock-if-one-can-use-synchronizedthis
src_7130570cedc3b36	Locking Mechanism in Java - Medium	https://medium.com/@premchandu.in/locking-mechanism-in-java-3eaf0d28066c
src_b89ffc673100912	Reentrant Locks Concurrency Interview - AlgoMaster.io	https://algomaster.io/learn/concurrency-interview/reentrant-locks
src_131ea93a75ea15cd	Synchronized vs. Lock API ReentrantLock Java ... - YouTube	https://www.youtube.com/watch?v=hMjl-YiVAvE
src_bb64fdef97a4461e	Java's Multithreading Superpower: Mastering Reentrant Locks	https://medium.com/@deepu144/javas-multithreading-superpower-mastering-reentrant-locks-d5f30e792b26
src_7f3277de8aab8366	Why You Should Ditch Synchronized for ReentrantLock in Java	https://www.linkedin.com/posts/vinicius-hc-castro_java-concurrency-multithreading-activity-7381072458042404864-4gpJ
src_002248c8e27de13f	What is Reentrant lock in Java multithreading and how it works?	https://www.quora.com/What-is-Reentrant-lock-in-Java-multithreading-and-how-it-works
src_4948fce5585ab120	ReentrantLock in Java - GeeksforGeeks	https://www.geeksforgeeks.org/java/reentrant-lock-in-java/
src_3578259fb9d68ca	Java Locks Explained: When to Use Synchronized, ReentrantLock ...	https://medium.com/@priyaranjanpatraa/java-locks-explained-when-to-use-synchronized-reentrantlock-and-readwritelock-d0bf83c14370
src_7edd9308f0e7a3d8	ReentrantLock (Java Platform SE 8) - Oracle Help Center	https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/locks/ReentrantLock.html
src_85f77976864f1d4f	embeddedmonologue - mutual exclusion and synchronization	https://sites.google.com/site/embeddedmonologue/home/mutual-exclusion-and-synchronization
src_2273c72bdae2fa66	Java Synchronization (Mutual Exclusion) Benchmark	https://baptiste-wicht.com/posts/2010/09/java-synchronization-mutual-exclusion-benchmark.html
src_087e8ed8bef1468f	Mutual Exclusion	https://cs.lmu.edu/~ray/notes/mutualexclusion/
src_9fd76c9333cb41cd	Why synchronized Is the Least Interesting Thing About Java ...	https://medium.com/javarevisited/why-synchronized-is-the-least-interesting-thing-about-java-concurrency-ca8f0252e534
src_b46de0325c9c48d4	Mutual Exclusion (Mutex)	https://stevengong.co/notes/Mutual-Exclusion
src_2018af3ee5c3ddcd	java - Mutual exclusion - How synchronized works? - Stack Overflow	https://stackoverflow.com/questions/47851595/mutual-exclusion-how-synchronized-works

Source id	Title	URL
src_40c7007a36456896	Synchronizing Threads like a Pro: Dive into Mutual Exclusion and ...	https://medium.com/@amitvsolutions/synchronizing-threads-like-a-pro-dive-into-mutual-exclusion-and-cooperation-f9a89c802d09
src_dd392efea07f76aa	[PDF] critical sections, mutual exclusion, test-and-set, spinlocks, blocking ...	https://student.cs.uwaterloo.ca/~cs350/S19/notes/synchronization-1up.pdf
src_f4296e78b7e24c91	CPU Scheduling, Mutual Exclusion & More - Ep 38 - YouTube	https://www.youtube.com/watch?v=mTzqVinpU1E
src_d4f6d727a936351a	Is not this code violating mutual exclusion in critical section?	https://stackoverflow.com/questions/27840116/is-not-this-code-violating-mutual-exclusion-in-critical-section
src_2769bca69a673ca4	Java concurrency — fixing races with AtomicReference - OpenLiberty.io	https://openliberty.io/blog/2021/09/21/building-on-atomic-reference.html
src_f6441b13825e2c84	Why do we need to compareAndSet when set is already atomic in ...	https://stackoverflow.com/questions/37716360/why-do-we-need-to-compareandset-when-set-is-already-atomic-in-java
src_7652a1dbfa212dea	What are AtomicInteger and AtomicLong in Java? How it works? compareAndSet() Example Tutorial	https://javarevisited.blogspot.com/2023/09/what-is-atomicinteger-and-atomiclong-in.html
src_28270f7aa71cfc08	Mastering Atomic Variables — AtomicInteger, AtomicLong, and ...	https://medium.com/@umeshcapg/java-concurrency-programming-interview-mastering-atomic-variables-atomicinteger-atomiclong-7ed38834d4ce
src_97918ec6a2faa23c	How and when should we use `AtomicReference` in Java? - Ask a Question - TestMu AI Community	https://community.lambdatest.com/t/how-and-when-should-we-use-atomicreference-in-java/36508
src_e392c59762fc74d9	Lock-Free Programming with Atomic Variables CodeSignal Learn	https://codesignal.com/learn/courses/java-concurrency-foundations/lessons/lock-free-programming-with-atomic-variables
src_ce65d0b6020aa159	When to use AtomicReference (Java)? Is it really necessary?	https://stackoverflow.com/questions/2932505/when-to-use-atomicreference-java-is-it-really-necessary
src_f84992dea53f7dc7	How to use AtomicInteger in Java for thread-safe updates - LinkedIn	https://www.linkedin.com/posts/meheedihasaan_java-concurrency-multithreading-activity-7357124321326108672-8hMW
src_b192aaeabc9af54d	How to use Atomic classes for safe thread operations in Java	https://www.linkedin.com/posts/avinashkr91_java-concurrency-multithreading-activity-7388473615035486208-52OC
src_336849134e7cddc2	Thread Safety in Kotlin: Demystifying Synchronization, Volatile, and ...	https://proandroiddev.com/thread-safety-in-kotlin-demystifying-synchronization-volatile-and-atomicreference-in-android-3f7cfab78404
src_6498120ca7234d4c	How Java's Atomic Variables Work Without Locks - Medium	https://medium.com/@AlexanderObregon/the-mechanics-behind-how-javas-atomic-variables-guarantee-thread-safety-without-locks-719cad1ab9dd
src_4a54a577ff2897ca	An Introduction to Atomic Variables in Java - Baeldung	https://www.baeldung.com/java-atomic-variables
src_0ba7b71a1e9d9277	This article delves into the trade-offs of using #synchronized ...	https://www.facebook.com/61555399162262/posts/this-article-delves-into-the-trade-offs-of-using-synchronized-methods-and-synchr/122206781744179972/
src_a6c5fc46d5131c28	Changelog - The Linux Kernel Archives	https://cdn.kernel.org/pub/linux/kernel/v7.x/ChangeLog-7.0
src_fe917a8f5ee3cf19	https://www.microsoft.com/releascommunications/api/v2/m365?responseFormat=csv	https://www.microsoft.com/releascommunications/api/v2/m365?responseFormat=csv
src_73871fd9d6497af9	[PDF] The American Ceramic Society Spring Meeting 2026 ABSTRACT ...	https://ceramics.org/wp-content/uploads/2026/03/3.23.SPRING26_Abstracts-3-002.pdf
src_aecd1d7b7f359978	High-frequency applications of high-temperature superconductor thin films - IOPscience	https://iopscience.iop.org/0034-4885/65/10/201/pdf/0034-4885_65_10_201.pdf
src_cd41c5dbe01d4561	Java Performance: synchronized() vs Lock - Reddit	https://www.reddit.com/r/java/comments/i1vby/java_performance_synchronized_vs_lock/
src_d1af9447c6a833be	java - is reentrant lock is complete replacement for synchronisation?	https://stackoverflow.com/questions/39961031/is-reentrant-lock-is-complete-replacement-for-synchronisation
src_23ad02389588f278	Understand Synchronizers in Java Concurrency Library In Depth[2]	https://anytech.medium.com/reentrantunderstand-synchronizers-in-java-concurrency-library-in-depth-2-reentrantreadwritelock-04375aa9c091
src_e9cf77bc8f2e1b08	How the Humble Synchronized beats ReentrantLock.. Simplified for a long suffering Java Programmer	https://medium.com/@apusingh1967/how-synchronized-beats-reentrantlock-simplified-for-a-long-suffering-java-programmer-c83d069a6fc0
src_b0c8b52b0589abac	Why is ReentrantLock recommended over Synchronized for ... - wãNN	https://www.zhihu.com/en/answer/3417731035
src_6db2822a8ddc339	Using ReentrantLock instead of synchronized for improving ...	https://stackoverflow.com/questions/33689850/using-reentrantlock-instead-of-synchronized-for-improving-performance
src_ea34e5119cd65f08	java.util.concurrent.locks.ReentrantLock generates a lot of garbage	https://bugs.openjdk.org/browse/JDK-6911759
src_cd48511284e57e4d	Deadlocks and Lock Mechanisms CodeSignal Learn	https://codesignal.com/learn/courses/java-concurrency-foundations/lessons/deadlocks-and-lock-mechanisms
src_bc2a35136b	Java Synchronized vs ReentrantLock - AnyTech -	https://anytech.medium.com/java-synchronized-vs-reentrantlock-which-lock

Source id	Title	URL
d3f248	Medium	-should-i-use-really-0cd8bd7cd3e5
src_94e72723b4f1ff2e	Java Concurrency Unlocked: A Comparative Guide to ... - Unlogged	https://www.unlogged.io/post/java-concurrency-unlocked-a-comparative-guide-to-synchronization-tools

Trust score — methodology & session summary

Each verified claim receives a headline trust score from 0 (low) to 100 (high). In the agent pipeline, evidence for synthesis is labeled HIGH, MODERATE, or LOW from that score: HIGH when score >= 81, MODERATE when 51–80, LOW when 0–50.

The headline score is a weighted blend of five dimensions (see per-claim table below when available): source_count (weight 15%) — more cited sources increase the score up to a cap; source_authority (25%) — higher for strong domains (.gov, .mil, .edu, Wikipedia, academic and major hubs); source_agreement (15%) — neutral placeholder until an entailment model is added; recency (15%) — newer publication dates score higher; fact_checker (30%) — score from the verification pass on claim–source alignment. If a claim has no sources, the module uses a reduced, fact-checker-heavy formula with a low cap.

This session: 1 verified claim(s). Mean headline score: 62.0/100. Bands: HIGH 0, MODERATE 1, LOW 0.

Per-claim score components (0–100 each, before weighting)

Band	Headline	src_cnt	authority	agree	recency	fact_chk	Claim (excerpt)
MODERATE	62	35	88	70	60	50	Research note tied to retrieved source [src_b9e218c7f2b8eca5] for «What are the main trade-offs between synchronized blocks vs explicit locks in java?»: Even while t is in its synchronized block, another thread can dange